Pando: The Trust Enabler
Bridging Payments, Deliveries, and Cash in E-Commerce

Version
v1.1

PONDO Integration Guide

Fintech Checkout Orchestration Layer

Amazon South Africa · Takealot · E-Commerce Partners

April 2026
Contributor: Lebo Mpeta

📖 Overview

PONDO is a **payment gateway checkout layer** designed for seamless integration into e-commerce platforms such as **Amazon SA** and **Takealot**. PONDO acts as a trusted checkout orchestration layer between the partner cart, payment authorisation, customer verification, Payment Enabled Delivery (PED), sponsor reporting, and reconciliation.

This guide provides technical steps, sandbox access, API contracts, SDK patterns, webhook callbacks, and compliance notes to help engineering teams incorporate PONDO into their environments with minimal checkout changes.

Recommended Integration Flow

1. Partner creates a PONDO payment intent for an order.
2. PONDO returns an intent token and hosted or embedded checkout state.
3. Customer confirms payment, consent, and delivery details.
4. Partner receives synchronous confirmation for the checkout session.
5. PONDO sends asynchronous webhook updates for settlement, delivery, refund, and reconciliation events.
6. Partner stores the PONDO transaction reference against its internal order ID.

Environments

ENVIRONMENT	PURPOSE	BASE URL
Sandbox	Partner engineering, QA, fake payments, webhook testing	https://sandbox-api.pondo.example.com
Preview	Vercel preview deployments for review builds	Active Vercel preview URL
Production	Live checkout traffic	https://api.pondo.example.com

🚀 Quick Start

1. INSTALL SDK

```
npm install @pondo/checkout-sdk
```

2. INITIALIZE

```
import { Pondo } from "@pondo/checkout-sdk";

const pondo = new Pondo({
  apiKey: process.env.PONDO_API_KEY!,
  environment: "sandbox",
  partnerId: "takealot"
});
```

3. CREATE PAYMENT INTENT

```
const intent = await pondo.createPaymentIntent({
  orderId: "TAK-10004567",
  amount: 18999.00,
  currency: "ZAR",
  customer: {
    id: "cust_123",
    email: "customer@example.com"
  }
});

return intent.checkoutUrl;
```

4. BROWSER HANDOFF

```
// Redirect customer to PONDO checkout
window.location.href = intent.checkoutUrl;

// Or use React component:
async function startCheckout() {
  const res = await fetch("/api/checkout/pondo-intent",
    { method: "POST",
      body: JSON.stringify({ orderId }) }
  );
  const intent = await res.json();
  window.location.href = intent.checkoutUrl;
}
```

> ⚠️ **Security Note**
> Never expose secret API keys in browser code. Browser integrations must call a partner backend, which then calls PONDO.

🔐 Authentication

Sandbox uses Bearer API keys. Production supports OAuth2 client credentials or signed API keys with rotation and expiry.

```
POST /v1/payment-intents
Authorization: Bearer pondo_sandbox_xxx
X-PONDO-Partner-Id: amazon-sa
Idempotency-Key: order-12345-create-payment-intent
Content-Type: application/json
```

✅ PRODUCTION AUTH

- OAuth2 client credentials (server-to-server)
- Signed API keys with rotation & expiry
- Partner-level permissions enforced
- Idempotency-Key required on mutations

🧪 SANDBOX AUTH

- API key: pondo_sandbox_xxx
- No OAuth2 required
- Test cards provided on request
- Webhook signing still enforced

📋 API Contract

OpenAPI 3.1 spec published at `api/openapi/pondo-checkout.v1.yaml`

Create Payment Intent

Request

```
{
  "partner": "takealot",
  "orderId": "TAK-10004567",
  "amount": 18999.00,
  "currency": "ZAR",
  "customer": {
    "id": "cust_123",
    "email": "customer@example.com",
    "phone": "+27821234567"
  },
  "items": [{
    "sku": "SAMSUNG-65-QLED",
    "name": "Samsung 65 QLED TV",
    "quantity": 1,
    "unitAmount": 18999.00
  }],
  "redirectUrls": {
    "success": "https://partner.example.com/
success",
    "cancel": "https://partner.example.com/c
ancel"
  }
}
```

Response

```
{
  "id": "pi_01HTZPONDO123",
  "status": "requires_confirmation",
  "token": "pit_01HTZSECURETOKEN",
  "checkoutUrl": "https://checkout.pondo.exa
mple.com/
    intents/pi_01HTZPONDO123",
  "expiresAt": "2026-04-30T18:00:00Z"
}
```

Confirm Payment Response

```
{
  "id": "pi_01HTZPONDO123",
  "orderId": "TAK-10004567",
  "status": "authorized",
  "pondoReference": "PONDO-20260430-000123",
  "qrPayload": "pondo://payment/PONDO-202604
30-000123"
}
```

Payment Status Codes

STATUS	MEANING
`requires_confirmation`	Intent created, customer has not completed PONDO confirmation
`authorized`	Payment authorized — order can continue
`processing`	Payment/delivery/reconciliation workflow is active
`settled`	Payment settled
`reconciled`	Sponsor and partner reporting completed
`cancelled`	Customer or partner cancelled before authorisation
`failed`	Payment, KYC, risk, or system failure
`refunded`	Full refund completed

Webhooks

Partners expose a webhook endpoint to receive asynchronous state changes. PONDO signs every request.

```
POST https://partner.example.com/webhooks/pondo
X-PONDO-Signature: t=1714488600,v1=<hmac>
X-PONDO-Event-Id: evt_01HTZEVENT123
Content-Type: application/json

{
  "id": "evt_01HTZEVENT123",
  "type": "payment_intent.authorized",
  "createdAt": "2026-04-30T14:50:00Z",
  "data": {
    "paymentIntentId": "pi_01HTZPONDO123",
    "orderId": "TAK-10004567",
    "status": "authorized",
    "pondoReference": "PONDO-20260430-000123"
  }
}
```

WEBHOOK REQUIREMENTS

- ✓ Verify X-PONDO-Signature before processing
- ✓ Return 2xx only after event safely stored
- ✓ Treat events as at-least-once delivery
- ✓ Use X-PONDO-Event-Id for idempotency
- ✓ Handle out-of-order events by checking intent status

EVENT	PURPOSE
`payment_intent.created`	Intent accepted
`payment_intent.authorized`	Order may proceed
`payment_intent.failed`	Show failure route
`payment_intent.settled`	Funds settled
`refund.created`	Refund accepted
`refund.succeeded`	Refund complete

Error Model

```
{
  "error": {
    "code": "invalid_request",
    "message": "Amount must be greater than zero.",
    "requestId": "req_01HTZREQUEST123",
    "details": { "field": "amount" }
  }
}
```

CODE	HTTP	MEANING
`invalid_request`	400	Missing or invalid field
`unauthorized`	401	Missing or invalid credentials
`forbidden`	403	Partner not allowed to perform this action
`not_found`	404	Intent, refund, or order not found
`idempotency_conflict`	409	Same idempotency key used with different payload
`risk_declined`	402	KYC, credit, fraud, or payment risk declined
`rate_limited`	429	Partner exceeded allowed request rate
`internal_error`	500	Unexpected PONDO service error

🧪 Sandbox Environment

DEMO PORTAL

Vercel Deployment — obtain URL from integration contact.

`sandbox-api.pondo.example.com`

TEST CARDS

`4111 1111 1111 1111`

Visa — Success

`4000 0000 0000 0002`

Visa — Insufficient Funds

WEBHOOKS

Test endpoint:

```
POST /webhook/payment-status
```

Receives transaction updates in sandbox.

📦 Partner Onboarding Checklist

PONDO PROVIDES

- ✓ Partner ID and display name
- ✓ Sandbox API key or OAuth2 client credentials
- ✓ Webhook signing secret
- ✓ Allowed callback & webhook URLs
- ✓ IP allowlist requirements (if used)
- ✓ Sandbox test cards and customer profiles
- ✓ Integration support contact
- ✓ Escalation contact for payment & compliance

PARTNER PROVIDES

- ✓ Checkout architecture owner
- ✓ Backend integration owner
- ✓ Security/risk reviewer
- ✓ Webhook endpoint URL
- ✓ Order ID format and refund rules
- ✓ Production traffic estimate
- ✓ Go-live and rollback window

Integration Steps

1. **Install SDK**
   npm install @pondo/checkout-sdk or clone the SDK repository
2. **Configure API Keys**
   Use sandbox keys first. Never expose secret keys in the browser.
3. **Embed Checkout Flow**
   Use SDK createPaymentIntent + redirect to checkoutUrl
4. **Test in Sandbox**
   Use provided test cards. Verify webhook events are received and idempotency works.
5. **Security Review**
   Complete PCI DSS checklist. Confirm TLS 1.2+, tokenisation, and audit log access.

6

Switch to Production

Swap sandbox keys for production. Confirm partner IDs and go-live window.

Compliance & Security

PCI DSS ALIGNMENT

- ✓ Card data never stored; tokenisation enforced
- ✓ All card interactions handled server-side
- ✓ No raw card numbers passed to partner systems

DATA ENCRYPTION

- ✓ TLS 1.2+ enforced for all API calls
- ✓ AES-256 encryption for data at rest
- ✓ Signed webhook payloads (HMAC-SHA256)

AUDIT & LOGGING

- ✓ Audit logs available via /api/logs endpoint
- ✓ Immutable event log for all state transitions
- ✓ Request IDs on every API response for tracing

POPIA COMPLIANCE

- ✓ Consent captured at confirmation step
- ✓ popiaAccepted and termsAccepted required
- ✓ Customer data handling per POPIA regulations

CI/CD Pipeline

GitHub Actions pipeline triggered on every commit to `main`:

1. Build & test SDK
2. Deploy sandbox portal to Vercel
3. Publish updated API docs
4. Tag release (v1.0, v1.1) for dependency locking

VERSIONING

- Releases tagged (v1.0, v1.1) for dependency locking
- Changelog maintained in CHANGELOG.md
- CI/CD ensures reproducible builds
- Preview deployments on every pull request

REPO

`github.com/shelod-eng/PONDO`